

DAY 08: QUARKUS FUNDAMENTALS

Data Bytes Technologies · Java 25 & Modern Stack

Today the reference app is **born**. By tonight you have a real, running web service, and you understand every line of it.

// WHERE WE ARE

The toolbox is full. Now we assemble it into one running system.

Week 1: foundations

- Day 1: Git
- Day 2: Docker & Compose
- Day 3: Java for polyglots
- Days 4–5: Java concurrency

Week 2: the running system

- Day 6: PostgreSQL, via `psql`
- Day 7: Kafka, from the CLI
- **Day 8: Quarkus: build the app** ← you are here
- Day 9: give the app a database
- Day 10: messaging + capstone

💡 Days 6–7 you learned to talk to Postgres and Kafka **directly**. Today we build the **app** that will tie them together, starting with just the web layer.

// WHY BUILD THE APP NOW, AND WHY START SMALL?

You've seen the pieces. A service is what makes them cooperate.

- The **KYC Service** is the spine of the rest of the course: every later day grows *this* code.
- Today it does one job well: **register an applicant over HTTP and answer cleanly**. No database yet.
- We grow it commit by commit, so each change is small enough to read, review, and trust.

Starting small is a feature, not a limitation: you'll understand the whole thing before it gets big.

// BY THE END OF TODAY, YOU CAN...

- explain **what a framework does for you**, and what Quarkus does differently
- describe **dependency injection** in plain terms, and why you stop writing `new`
- scaffold a service with the `quarkus` CLI and live in **dev mode**
- expose a **validated REST endpoint** that returns clean JSON, and map errors to tidy responses
- configure **one app for three environments**, and add a **health check**

// FIRST: WHAT DOES A FRAMEWORK EVEN DO FOR US?

You *could* write a web service from raw sockets. You won't, because the same plumbing recurs in every service ever written.

- accept an HTTP connection, **route** a URL to a method, parse and serialise **JSON**
- **wire your objects together**, read **configuration**, manage transactions
- expose **health**, metrics, and logs the platform can read

A **framework** is that plumbing (done once, well) so you write only the part that is *your application*.

// DEPENDENCY INJECTION, IN PLAIN TERMS

💡 **In plain terms:** you stop writing `new` for your collaborators. You *declare* what you need, and the framework hands it to you, already built and wired.

```
// With DI: you ask for it; the container builds one shared instance and hands it over.  
@Inject  
ApplicantService service;
```

Same object graph, but you no longer *assemble* it. You say *what* you need; the container works out *how* to build it.

// ONE PICTURE: WHO BUILDS WHAT



The container owns the **lifecycle**: it builds these once, as shared singletons, and hands the same instances wherever they're `@Inject`-ed.

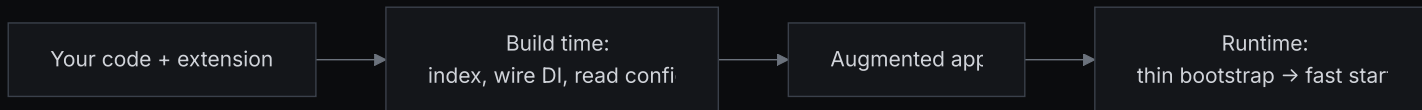
// WHY DI IS WORTH IT

- **Less wiring:** no hand-assembled object graphs duplicated across the codebase.
- **One place to swap** an implementation: in-memory today, a real database tomorrow.
- **Testable:** hand a **fake** collaborator to a unit test instead of the real thing.

💡 The payoff you'll *feel* on Day 9: because `ApplicantResource` only asks for an `ApplicantService`, we can swap the store under it (from a `Map` to PostgreSQL) without touching the web layer.

// QUARKUS'S BIG IDEA: DO THE WORK AT BUILD TIME

Most frameworks do their setup when the app **starts**: scan the classpath, build the DI graph, read config. Quarkus does as much of that as possible while it **compiles**.



The framework work doesn't vanish. It moves **earlier**, into the build. So startup is tiny.

// WHY SHOULD YOU CARE ABOUT FAST BOOT?

💡 **In plain terms:** a service that boots in ~30 ms and sips memory can be scaled up, scaled to **zero**, and restarted freely. That's what makes container-and-cloud deployment cheap.

Coming from Spring? The model feels familiar (inject this, annotate that) but Quarkus has already done the heavy lifting by the time the process starts. You'll see it boot in a moment.

// SCAFFOLDING: NEVER COPY A POM.XML

We generate the project with the `quarkus` CLI, choosing only the capabilities ("extensions") we need today:

```
quarkus create app com.databytes:kyc-service \
  --extension=rest-jackson,hibernate-validator,smallrye-health
```

- `rest-jackson`: **Quarkus REST** + JSON (the modern REST extension)
- `hibernate-validator`: declarative input validation (Bean Validation)
- `smallrye-health`: `/q/health` liveness & readiness probes

Each extension is a capability with its build-time magic baked in. We add more **only when we need them**: persistence tomorrow, messaging Day 10.

// CODE ALONG: SCAFFOLD AND LAUNCH

 **With me.** Generate the project, then start it in dev mode:

```
quarkus create app com.databytes:kyc-service \
  --extension=rest-jackson,hibernate-validator,smallrye-health
cd kyc-service
quarkus dev          # or: ./mvnw quarkus:dev
```

Wait for the banner, then open <http://localhost:8080/q/dev/>, Quarkus' Dev UI. Notice how fast it came up. **Leave it running:** we edit code and watch it reload.

// A TOUR OF WHAT GOT GENERATED

- `src/main/java/...` : **your code**: resources, services, domain
- `src/main/resources/application.properties` : **configuration**
- `src/test/java/...` : **tests** (we lean on these tomorrow)
- `pom.xml` : dependencies, versions pinned by the **Quarkus BOM**
- `src/main/docker/Dockerfile.*` : container builds, for later

A small, predictable layout. You always know where a thing goes, which matters more than it sounds when a whole team shares one codebase.

// DEV MODE IS YOUR SUPERPOWER

`quarkus dev` is not just "run the app". It gives you a tight feedback loop:

- **Live reload:** save a file, the *next request* uses the new code. No restart.
- **Continuous testing:** affected tests re-run as you type; press `r` to run all.
- the **Dev UI** at `/q/dev/` to inspect beans, config, and endpoints.

💡 You will live in this loop all week: edit → re-request → see the result. No build-restart cycle.

// TRY IT NOW (2 MIN): AN ENDPOINT YOU DIDN'T WRITE

With dev mode still running, in another terminal:

```
curl -i localhost:8080/q/health
```

You should get `200` and `"status": "UP"`. You didn't write that endpoint. `smallrye-health` did. **Extensions earn their keep.**

Now open `/q/dev/` in the browser and find the **Beans** panel. There's your `ApplicantResource` and the CDI graph, live.

// HOW AN HTTP REQUEST WILL FLOW

Before we write code, the shape. Three small pieces, each easy to test on its own:

// THE REQUEST IS A RECORD (YOU KNOW THESE)

The body of `POST /applicants` arrives as an immutable **record**, and the validation rules live right on its components:

```
public record RegisterApplicantRequest(  
    @NotBlank String fullName,  
    @NotBlank String nationalId,  
    @NotNull @Past LocalDate dateOfBirth,  
    @NotBlank String country,  
    @NotBlank @Email String email,  
    Map<String, String> attributes) {}
```

`@NotBlank`, `@Past`, `@Email` are **Bean Validation**: they *describe* the rules; the framework *enforces* them. No `if (email == null) throw ...` scattered through your code.

// 🖥️ CODE ALONG: THE DTO RECORDS

🖥️ Type these with me. Two records: what comes in, and what goes out.

```
public record RegisterApplicantRequest(
    @NotBlank String fullName,
    @NotBlank String nationalId,
    @NotNull @Past LocalDate dateOfBirth,
    @NotBlank String country,
    @NotBlank @Email String email,
    Map<String, String> attributes) {}

public record ApplicantResponse(
    Long id, String fullName, String nationalId, LocalDate dateOfBirth,
    String country, String email, String status,
    Map<String, String> attributes, Instant createdAt) {

    public static ApplicantResponse from(Applicant applicant) {
        return new ApplicantResponse(applicant.id, applicant.fullName, applicant.nationalId,
            applicant.dateOfBirth, applicant.country, applicant.email, applicant.status.name(),
            applicant.attributes, applicant.createdAt);
    }
}
```

// 🖱️ CODE ALONG: THE REST RESOURCE

🖱️ With me. The front door, Quarkus REST + JAX-RS annotations:

```
@Path("/applicants")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ApplicantResource {

    @Inject ApplicantService service;

    @POST
    public Response register(@Valid RegisterApplicantRequest request, @Context UriInfo uriInfo) {
        Applicant draft = new Applicant();
        draft.fullName = request.fullName();
        draft.nationalId = request.nationalId();
        draft.dateOfBirth = request.dateOfBirth();
        draft.country = request.country();
        draft.email = request.email();
        Applicant saved = service.register(draft);
        URI location = uriInfo.getAbsolutePathBuilder().path(String.valueOf(saved.id)).build();
        return Response.created(location).entity(ApplicantResponse.from(saved)).build();
    }
}
```

// WHAT DOES @VALID ACTUALLY DO?

💡 In plain terms: `@Valid` tells Quarkus: *before you call my method, check the rules on this object*. If anything fails, my method **never runs**. A validation error is raised instead.

So your handler only ever sees **good** input. The "what if it's garbage?" question is answered in **one** place, declaratively, not scattered through your logic.

// TRY IT NOW (3 MIN): GOOD APPLICANT VS BAD APPLICANT

Register a valid applicant, then an invalid one, and compare the responses:

```
curl -s localhost:8080/applicants -H 'content-type: application/json' \
  -d '{"fullName":"Ada Lovelace","nationalId":"NID-1","dateOfBirth":"1990-01-01","country":"GB","email":"ada@exam

curl -s localhost:8080/applicants -H 'content-type: application/json' \
  -d '{"fullName":"","nationalId":"NID-1","dateOfBirth":"2999-01-01","country":"GB","email":"not-an-email"}' | jq
```

// TURN UGLY ERRORS INTO TIDY ONES

By default a validation failure is a wall of stack trace. We map it to a clean, predictable JSON body with an

`ExceptionHandler`:

```
// With our ExceptionMapper: clean, structured, 400
{
  "status": 400,
  "error": "Bad Request",
  "message": "Request validation failed",
  "violations": [ { "field": "email", "message": "must be a well-formed email address" } ]
}
```

One small `@Provider` class converts *any* validation failure into this shape. The same idea gives a missing applicant a clean **404**.

// 🖥️ CODE ALONG: THE EXCEPTION MAPPERS

🖥️ With me. One `@Provider` turns validation failures into a tidy `400`:

```
@Provider
public class ValidationExceptionHandler implements ExceptionMapper<ConstraintViolationException> {

    @Override
    public Response toResponse(ConstraintViolationException exception) {
        List<ApiError.FieldViolation> violations = exception.getConstraintViolations().stream()
            .map(v -> new ApiError.FieldViolation(lastNode(v), v.getMessage()))
            .toList();
        ApiError error = new ApiError(400, "Bad Request",
            "Request validation failed", violations, Instant.now());
        return Response.status(400).entity(error).build();
    }
    // lastNode(...) trims the path to just the field name, e.g. "email"
}
```

// 🖐️ EVERYONE WITH ME?

You should have, right now, a service that:

- returns **201** with a `Location` header for a good applicant
- returns **400** with field-level reasons for a bad one
- returns **404** for an unknown id
- reports **UP** at `/q/health`

If any of those isn't true, flag it now. The rest of the day (and tomorrow) builds directly on this.

// ONE FILE, THREE ENVIRONMENTS

A service runs in **dev**, in **tests**, and in **production**, with different settings. Quarkus keeps them in one file, prefixed by profile:

```
quarkus.application.name=kyc-service
quarkus.http.port=8080

%dev.quarkus.log.console.json=false      # human-readable logs while developing
%prod.quarkus.log.console.json=true      # structured JSON logs in production
```

`%dev` / `%test` / `%prod` **override** the base. No separate config files to drift apart. The environment picks the profile.

// A TINY CONFIG STORY WORTH KNOWING

💡 In plain terms: `%test.quarkus.http.test-port=0` tells tests to grab **any free port**. Without it, the shared dev infra (which uses 8081, the conventional test port) would collide with your tests.

Small settings like this are how a project stays friction-free for a whole team. We'll meet a few more as the app grows.

// HEALTH CHECKS: HOW THE PLATFORM KNOWS YOU'RE ALIVE

`smallrye-health` exposes probes the platform polls:

- **liveness** (`/q/health/live`): "is the process healthy, or should it be **restarted**?"
- **readiness** (`/q/health/ready`): "can it take traffic **right now**?"

💡 **In plain terms:** Kubernetes (and our Compose file) call these to decide whether to send you requests or restart you. You expose the **truth**; the platform **acts** on it.

// TRY IT NOW (2 MIN): READ THE PROBES

```
curl -s localhost:8080/q/health/live | jq  
curl -s localhost:8080/q/health/ready | jq
```

Both should report `"status": "UP"`. Now stop dev mode (`q` in the dev console) and curl again. The connection refused is the liveness signal a platform would act on.

// COMMON TRAPS

- **Forgot `@Valid`**: your handler runs on garbage input. Validation only fires when you ask for it.
- **Returning the domain object from the resource**: keep HTTP types (`ApplicantResponse`) separate from the domain. It saves real pain when the model and the API need to diverge.
- **Expecting a restart**: in dev mode you don't restart; just save and re-request.
- **Hand-editing the `pom.xml`**: add capabilities with `quarkus extension add`, not by hand.

// RECAP, IN PLAIN TERMS

Today you built a real service from a few small, testable pieces:

- a **framework** owns the plumbing; **DI** wires your objects so you stop writing new
- **Quarkus** does the work at **build time** → fast boot, and a great **dev mode** loop
- a **record** carries validated input; a **resource** speaks HTTP and nothing else
- a **service** holds the logic (an in-memory store for now)
- a **mapper** makes errors clean; **profiles** and **health** are the habits every service needs

// TODAY'S LAB

✎ Scaffold the KYC Service yourself with the `quarkus` CLI, add the validated `POST` / `GET` endpoints over an in-memory store, wire the exception mappers, and live in dev mode + continuous testing.

Brief: `labs/day-08/` · Solution to check against: `git checkout checkpoint/day-08`

Acceptance: `201` / `400` / `404` behave, `/q/health` is `UP`, and `./mvnw -B verify` is green.

// TAKEAWAY

A modern service is a few small, single-purpose pieces the framework wires together, so each one stays simple, testable, and easy to change.

Tomorrow, Day 9: those applicants stop living in a `HashMap` and move into **PostgreSQL**, with a real schema, Flyway migrations, and Panache. And because of DI, the web layer won't change a line.